

Neptun kód: CRGYRX Név: Wang Zhibo  
Beadás verziószáma: 1.  
Később kiegészítéssel(!)  
2. (javított Uf...)  
3. (javított Algoritmus)

## Feladat

### Gazdaságos üzemeltetés

A Budapest-Székesfehérvár vasútvonalon egy vonat kalauza minden állomáson feljegyezte, hogy hányan szálltak fel a vonatra, illetve hányan szálltak le. (Budapesten biztos nincs leszálló, Székesfehérváron biztos nincs felszálló, aki leszállt, az nem száll vissza.)

Készíts programot, amely eldönti, hogy gazdaságos-e a vonat üzemeltetése, ha egy utasnak egy állomásnyi távolság  $N$  Ft-ba kerül, a vonat egy állomásnyi útja pedig  $M$  Ft-ba kerül!

#### Bemenet

A standard bemenet első sorában az állomások száma van ( $1 \leq \text{állomás-szám} \leq 1000$ ), második sorában az egy állomásnyi távolság ára személyenként ( $0 < N \leq 100$ ) és az egy állomásnyi vonatút költsége ( $0 < M \leq 100\ 000$ ), majd soronként szóközzel elválasztva az egyes állomásokon leszállók ( $0 \leq \text{leszállók} \leq 800$ ) és felszállók ( $0 \leq \text{felszállók} \leq 800$ ) száma.

#### Kimenet

A standard kimenet egyetlen sorába egyetlen egész számot kell írni, amely 1, ha gazdaságos az üzemeltetés, és 0, ha nem!

#### Példa

| Bemenet  | Kimenet |
|----------|---------|
| 6        | 1       |
| 100 1000 |         |
| 0 15     |         |
| 10 30    |         |
| 0 32     |         |
| 47 0     |         |
| 20 26    |         |
| 26 0     |         |

## Specifikáció

**Be:**  $\text{allomas} \in \mathbb{N}$ ,  $\text{ar} \in \mathbb{N}$ ,  $\text{vonatk} \in \mathbb{N}$ ,  $\text{allomasadat} \in \text{Reszletek}[1..\text{allomas}]$ ,  $\text{Reszletek} = (\text{le}:\mathbb{N} \times \text{fel}:\mathbb{N})$

**Sa:**  $\text{tbevetel} \in \mathbb{N}$

**Ki:**  $\text{gazd} \in \mathbb{N}$

**Fv:**  $\text{aktualisutas}:\mathbb{N} \rightarrow \mathbb{N}$ ,  $\text{aktualisutas}(\text{adottmegallo}) = \text{SZUMMA}(i=1..\text{adottmegallo}, \text{allomasa-} \text{dat}[i].\text{fel} - \text{allomasadat}[i].\text{le})$

**Ef:** -

**Uf:**  $\text{tbevetel} = \text{SZUMMA}(i=1..\text{allomas}, \text{aktualisutas}(i) * \text{ar})$  and

$\text{tbevetel} > \text{allomas} * \text{vonatk} = \text{false} \rightarrow \text{gazd} = 0$  and

$\text{tbevetel} > \text{allomas} * \text{vonatk} = \text{true} \rightarrow \text{gazd} = 1$

## Sablon

### Összegzés sablon

#### Feladat

Adott az egész számok egy  $[e..u]$  intervalluma és egy  $f:[e..u] \rightarrow H$  függvény. A  $H$  halmaz elemein értelmezett az összeadás művelet. Határozzuk meg az  $f$  függvény  $[e..u]$  intervallumon felvett értékeinek az **összegét**, azaz a  $\sum_{i=e}^u f(i)$  kifejezés értékét! ( $e > u$  esetén ennek az értéke definíció szerint a nulla elem)

#### Specifikáció

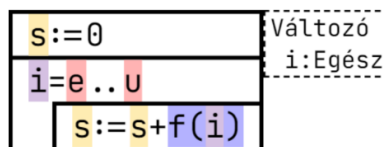
**Be:**  $e \in \mathbb{Z}$ ,  $u \in \mathbb{Z}$

**Ki:**  $s \in H$

**Ef:** -

**Uf:**  $s = \text{SZUMMA}(i=e..u, f(i))$

#### Algoritmus

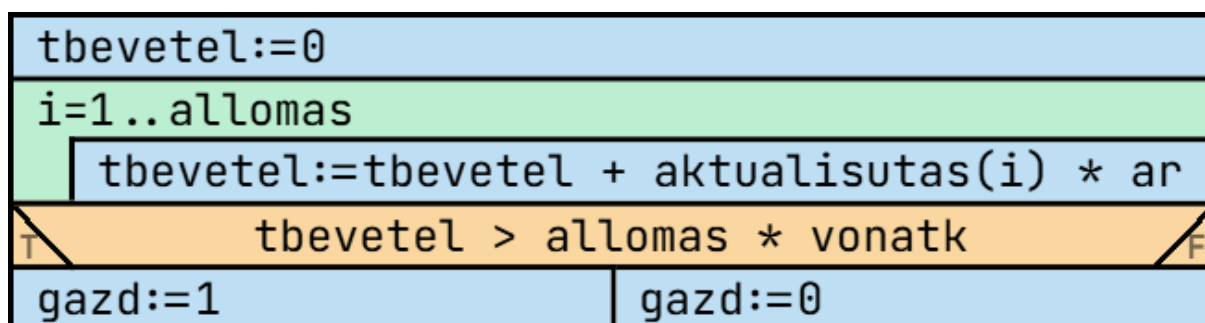


## Visszavezetés

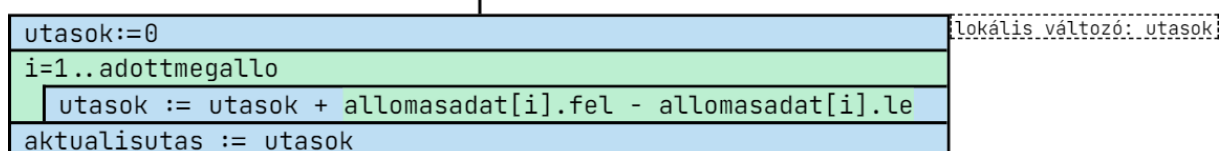
|           |                      |
|-----------|----------------------|
| összegzés |                      |
| e         | 1                    |
| u         | allomas              |
| s         | tbevetel             |
| f(i)      | aktualisutas(i) * ar |

|                                       |                                        |
|---------------------------------------|----------------------------------------|
| összegzés: aktualisutas(adottmegallo) |                                        |
| e                                     | 1                                      |
| u                                     | adottmegallo                           |
| s                                     | aktualisutas                           |
| f(i)                                  | allomasadat[i].fel - allomasadat[i].le |

## Algoritmus



aktualisutas(adottmegallo)



## Kód (C#)

// Wang Zhibo(CRGYRX) wangzhibo08@gmail.com

using System;

namespace \_2bead

{

internal class Program

{

struct Reszletek

{

public int le;

Megoldás sablon „Gazdaságos üzemeltetés” beadandó 1. fázishoz

```
public int fel;
}
static void Main(string[] args)
{
    int allomas, ar, vonatk, i, tbevetel, gazd;

    allomas = int.Parse(Console.ReadLine());
    string[] input = Console.ReadLine().Split(' ');
    ar = int.Parse(input[0]);
    vonatk = int.Parse(input[1]);

    Reszletek[] allomasadat = new Reszletek [allomas + 1];
    for (i = 1; i <= allomas; ++i)
    {
        string[] rekord = Console.ReadLine().Split(' ');
        allomasadat[i].le = int.Parse(rekord[0]);
        allomasadat[i].fel = int.Parse(rekord[1]);
    }

    tbevetel = 0;
    for (i = 1; i <= allomas; ++i)
    {
        tbevetel += aktualisutas(i, allomasadat) * ar;
    }

    if (tbevetel > allomas * vonatk)
        gazd = 1;
    else
        gazd = 0;

    Console.WriteLine(gazd);
}
```

```

}

static int aktualisutas (int adottmegallo, Reszletek[] allomasadat)
{
    int utasok, i;
    utasok = 0;
    for (i = 1; i <= adottmegallo; ++i)
    {
        utasok = utasok + allomasadat[i].fel - allomasadat[i].le;
    }
    return utasok;
}
}
}

```

## Bíró pontszám és képernyőkép

CHANGE TOPIC

SUBMIT

RESULT

DOWNLOAD

BACKUP

SUBMITTED

MAIN PAGE

EXIT/LOGIN

Result of the last submission

Total points: 100/

| Test# | Point | Verdict... | CPU time  |
|-------|-------|------------|-----------|
| 1.1   | 6/6   | Helyes     | 0.031 sec |
| 2.1   | 6/6   | Helyes     | 0.031 sec |
| 3.1   | 6/6   | Helyes     | 0.030 sec |
| 4.1   | 6/6   | Helyes     | 0.032 sec |
| 5.1   | 6/6   | Helyes     | 0.031 sec |
| 6.1   | 7/7   | Helyes     | 0.031 sec |
| 7.1   | 7/7   | Helyes     | 0.031 sec |
| 8.1   | 7/7   | Helyes     | 0.032 sec |
| 9.1   | 7/7   | Helyes     | 0.032 sec |
| 10.1  | 7/7   | Helyes     | 0.032 sec |
| 11.1  | 7/7   | Helyes     | 0.033 sec |
| 12.1  | 7/7   | Helyes     | 0.035 sec |
| 13.1  | 7/7   | Helyes     | 0.032 sec |
| 14.1  | 7/7   | Helyes     | 0.033 sec |
| 15.1  | 7/7   | Helyes     | 0.033 sec |

Date of submission: 2024-12-01 18:12:53.0

## Saját tesztfájlok

be1

6

100 1000

0 15

10 30

0 32

47 0

20 26

26 0

ki1

1

be2

4

20 1000

0 15

10 20

0 10

35 0

ki2

0

be3

3

10000 1000

0 0

0 0

0 0

ki3

0